Remote Job Finder — Build Spec for Claude Code

Oct 20, 2026 · @Abdullah Yaqub

Implementation spec for building the product end to end with Claude Code. Companion to the Product Spec; where they differ, this one wins.

1. How Claude Code should use this spec

Build milestone by milestone (section 13). Each milestone ends with its tests passing and its acceptance criteria checked before the next one starts.

First task in an empty repo: create `CLAUDE.md` at the root containing sections 2–3 in short form (principles, stack, commands, conventions), so every later session starts with them in context.

Hard rules (never break these):

- No paid or third-party data APIs and no LLM API calls from our server. All reasoning and crawling happens in the user's Claude. The only external service allowed is the payment provider (section 11).
- No scraping behind logins. No LinkedIn or Indeed crawling from our server. Respect robots.txt.
- No fabricated CV content. Server-side validation (section 8) is mandatory, not optional.
- Every personal row is scoped by `user_id`. No query reads personal data without it.

Conventions:

- Python 3.12, `uv`, `ruff` (lint + format), `mypy --strict`, `pytest`.
- TypeScript in strict mode, `pnpm` workspaces, ESLint, Vitest, Playwright.
- Migrations through Alembic only. The DDL in section 4 is the target state of migration 001.
- Settings come from environment variables through `pydantic-settings`. Never hard-code secrets.
- Use the latest stable version of each dependency at build time and pin it in the lockfile.

Ask the human before: adding a dependency not named here, changing a public tool schema, or changing the DDL.

Definition of done for any task:

- [ ] Tests written and passing; coverage of new code at 85% or more
- [ ] Lint, format, and type checks clean
- [ ] Tool or API docs updated
- [ ] UI changes checked in light and dark themes and at 360 px width
- [ ] No new warnings in the MCP Inspector for changed tools

2. Product summary and principles

The product is an MCP connector with interactive in-chat UI (MCP Apps) plus a small companion web app. The user's Claude crawls the web for remote jobs, our server dedupes them and scores ghost risk, and the user gets a ranked shortlist, a tailored CV, and a tracker without leaving Claude.

Two UI surfaces:

- **In-chat views (primary):** interactive HTML views rendered inside Claude next to the conversation, for the shortlist, job detail, CV studio, tracker, and profile review.
- **Web app (secondary):** landing page, sign-in, onboarding, billing, data controls, and a full-screen version of the tracker.

Principles every screen and tool follows:

1. **Chat-native first.** Anything doable by clicking is also doable by asking Claude. Views speed things up but never gate them.
2. **Evidence over vibes.** Every score shows its reasons and the source pages behind it. Every number links to where it came from.
3. **One decision per view.** Each view answers one question: which jobs, is this one real, is this CV right, where do I stand.
4. **Honest by design.** Ghost risk is always visible, gaps are named, and nothing is invented.
5. **Calm, not a feed.** A daily briefing of 10–25 jobs, ranked. No infinite scroll, no streaks, no dark patterns.
6. **Fast.** A view's first paint comes within 300 ms of the tool result arriving. Interactions respond within 100 ms, optimistically.
7. **Accessible.** WCAG 2.2 AA, full keyboard use, visible focus, `prefers-reduced-motion` respected, works at 360 px wide, follows the host's light or dark theme.

3. Tech stack and repo structure

The backend is Python with FastAPI, and the MCP server is mounted in the same app. The frontends are TypeScript and React in one pnpm workspace sharing a single design system.

Layer	Choice	Why
API + MCP server	FastAPI + official MCP Python SDK (Streamable HTTP)	One process, one auth layer
Database	Postgres 16 + SQLAlchemy 2 (async) + Alembic	jsonb for profiles, `pg_trgm` for fuzzy dedupe
Jobs queue	Celery + Redis (with beat for schedules)	CV compiles, signal recompute, nightly re-check
Auth	Self-hosted OAuth 2.1 authorization server (Authlib)	Required by Claude custom connectors; no third-party IdP
CV compile	Tectonic in a locked-down subprocess	Reproducible LaTeX, no shell escape
File storage	S3-compatible object storage	PDFs and .tex files
In-chat views	React + Vite + `@modelcontextprotocol/ext-apps`, each built to one self-contained HTML file	Served as `ui://` resources
Web app	Next.js (App Router)	Landing, onboarding, billing, settings
Design system	`packages/ui`: tokens + React components (Radix primitives, Tailwind)	Same look in chat views and web

```
remote-job-finder/
├── CLAUDE.md
├── docker-compose.yml          # postgres, redis, api, worker, web
├── backend/
│   ├── pyproject.toml
│   ├── alembic/
│   └── app/
│       ├── main.py             # FastAPI app, mounts /mcp and /oauth
```

```
|       ├── settings.py
|       ├── db/                 # models.py, session.py
|       ├── auth/               # oauth server, tokens, scopes
|       ├── mcp/
|       |   ├── server.py       # tool + resource registration
|       |   ├── tools/          # one module per tool
|       |   └── ui_resources.py # serves built view HTML as ui:// resources
|       ├── discovery/          # normalize.py, dedupe.py, eligibility.py
|       ├── ghost/              # signals.py, score.py
|       ├── cv/                 # schema.py, validate.py, render.py,
compile.py, templates/
|       ├── billing/
|       ├── workers/            # celery app + tasks
|       └── api/                # REST for the web app
|   └── tests/
├── frontend/
|   ├── pnpm-workspace.yaml
|   ├── packages/ui/            # tokens.css, components, icons
|   ├── packages/views/         # shortlist, job-detail, cv-studio, tracker,
profile-review
|   └── apps/web/               # Next.js
└── plugin/
    ├── .claude-plugin/plugin.json
    ├── skills/                 # daily-scan, fit-rubric, ghost-rubric, cv-
writer
    └── commands/               # jobs, apply, tracker, profile
```

The views build step writes `frontend/packages/views/dist/<view>.html`, which the backend reads at startup and serves as resources.

4. Data model

Jobs, companies, and sightings are shared across all users. Profiles, scores, CVs, and applications belong to one user. This is the target state of migration 001.

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS citext;

CREATE TABLE users (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  email         citext UNIQUE NOT NULL,
  name          text,
```

```
  country_code  char(2),                -- ISO 3166-1, drives eligibility
  plan          text NOT NULL DEFAULT 'free' CHECK (plan IN ('free','pro')),
  cv_quota_used int  NOT NULL DEFAULT 0,  -- reset monthly
  created_at    timestamptz NOT NULL DEFAULT now(),
  deleted_at    timestamptz
);

CREATE TABLE profiles (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  version     int  NOT NULL,
  data        bytea NOT NULL,           -- encrypted MasterProfile JSON
(section 8)
  preferences jsonb NOT NULL,           -- Preferences JSON (section 5)
  status      text NOT NULL CHECK (status IN
('proposed','active','rejected')),
  created_at  timestamptz NOT NULL DEFAULT now(),
  UNIQUE (user_id, version)
);
CREATE UNIQUE INDEX one_active_profile ON profiles(user_id) WHERE status =
'active';

CREATE TABLE companies (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name            text NOT NULL,
  name_norm       text NOT NULL,
  domain          text UNIQUE,
  ats_type        text CHECK (ats_type IN
('greenhouse','lever','ashby','workable','other')),
  ats_slug        text,
  careers_url     text,
  health_note     text,                 -- Claude's news summary: layoffs,
funding
  health_flag     text CHECK (health_flag IN
('growing','stable','shrinking','unknown')) DEFAULT 'unknown',
  health_checked_at timestamptz,
  created_at      timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX companies_name_trgm ON companies USING gin (name_norm
gin_trgm_ops);

CREATE TABLE jobs (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  company_id     uuid NOT NULL REFERENCES companies(id),
  title          text NOT NULL,
  title_norm     text NOT NULL,
  seniority      text CHECK (seniority IN
('intern','junior','mid','senior','staff','lead','unknown')),
  location_text  text NOT NULL,
  eligibility    text NOT NULL CHECK (eligibility IN
('worldwide','region_open','restricted','unclear')),
  allowed_countries char(2)[],           -- when region_open
  tz_min_overlap_hours numeric(3,1),
  remote_type    text NOT NULL CHECK (remote_type IN
('remote','hybrid','onsite')),
  employment_type text CHECK (employment_type IN
('full_time','contract','part_time','internship')),
  salary_min     int, salary_max int, salary_currency char(3), salary_period
text,
  posted_at      date,
  apply_url      text NOT NULL,
  on_company_ats boolean NOT NULL DEFAULT false,
  jd_text        text NOT NULL,
  jd_hash        char(64) NOT NULL,       -- sha256 of normalized JD
  status         text NOT NULL DEFAULT 'open' CHECK (status IN
('open','closed')),
  first_seen_at  timestamptz NOT NULL DEFAULT now(),
  last_seen_at   timestamptz NOT NULL DEFAULT now(),
  closed_at      timestamptz
);
CREATE UNIQUE INDEX jobs_dedupe ON jobs(company_id, title_norm, jd_hash);
CREATE INDEX jobs_open_recent ON jobs(status, last_seen_at DESC);
CREATE INDEX jobs_title_trgm ON jobs USING gin (title_norm gin_trgm_ops);

CREATE TABLE job_sightings (
  id          bigserial PRIMARY KEY,
  job_id      uuid NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,
  source_kind text NOT NULL CHECK (source_kind IN
('ats','careers_page','remote_board','search_result','recheck')),
  source_url  text NOT NULL,
  seen_at     timestamptz NOT NULL DEFAULT now(),
  submitted_by uuid REFERENCES users(id) ON DELETE SET NULL
);
CREATE INDEX sightings_job_time ON job_sightings(job_id, seen_at);

-- a repost = same company + title_norm, different jd_hash or a gap > 21 days
```

```
CREATE TABLE job_postings_history (
  company_id  uuid NOT NULL REFERENCES companies(id),
  title_norm  text NOT NULL,
  job_id      uuid NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,
  posted_at   date,
  PRIMARY KEY (company_id, title_norm, job_id)
);

CREATE TABLE ghost_signals (
  job_id            uuid PRIMARY KEY REFERENCES jobs(id) ON DELETE CASCADE,
  on_company_ats    boolean NOT NULL,
  repost_count_90d  int NOT NULL,
  age_days          int,
  salary_listed     boolean NOT NULL,
  company_health    text NOT NULL,
  base_score        int NOT NULL,          -- server part, 0-100
  reasons           jsonb NOT NULL,        -- [{code, points, detail,
source_url}]
  computed_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE job_scores (
  user_id         uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  job_id          uuid NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,
  profile_version int  NOT NULL,
  fit_band        text NOT NULL CHECK (fit_band IN
('match','stretch','reach','skip')),
  fit_summary     text NOT NULL,
  must_haves_met  jsonb NOT NULL,          -- [{requirement,
evidence_bullet_id}]
  gaps            jsonb NOT NULL,          -- [{requirement,
transferable_from, severity}]
  jd_quality_points int NOT NULL DEFAULT 0, -- Claude's part of ghost score
  ghost_score     int NOT NULL,             -- base + Claude part, clamped
  ghost_band      text NOT NULL CHECK (ghost_band IN
('low','medium','high')),
  dismissed       boolean NOT NULL DEFAULT false,
  scored_at       timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, job_id, profile_version)
);

CREATE TABLE cvs (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  user_id       uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  job_id        uuid REFERENCES jobs(id) ON DELETE SET NULL,
  profile_version int NOT NULL,
  template      text NOT NULL,
  content       jsonb NOT NULL,            -- CvContent JSON (section 8)
  tex_key       text NOT NULL,             -- object storage keys
  pdf_key       text,
  status        text NOT NULL CHECK (status IN
('queued','compiled','failed')),
  error         text,
  kit           jsonb,                      -- cover note, screening answers,
DM, gap report
  created_at    timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE applications (
  id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id      uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  job_id       uuid NOT NULL REFERENCES jobs(id),
  cv_id        uuid REFERENCES cvs(id) ON DELETE SET NULL,
  status       text NOT NULL CHECK (status IN
('saved','applying','applied','interviewing','offer','rejected','withdrawn','
ghosted')),
  applied_at   timestamptz,
  follow_up_at date,
  notes        text,
  updated_at   timestamptz NOT NULL DEFAULT now(),
  UNIQUE (user_id, job_id)
);
CREATE INDEX applications_followup ON applications(user_id, follow_up_at)
WHERE status = 'applied';

CREATE TABLE audit_log (
  id         bigserial PRIMARY KEY,
  user_id    uuid REFERENCES users(id) ON DELETE SET NULL,
  action     text NOT NULL,                -- tool name or api route
  meta       jsonb,
  at         timestamptz NOT NULL DEFAULT now()
);
```

OAuth tables (clients, authorization codes, tokens) follow Authlib's SQLAlchemy mixins and live in `auth/models.py`.

5. MCP server

The MCP server runs at `POST /mcp` over Streamable HTTP and needs an OAuth bearer token. It exposes 10 model-facing tools (the 10th, update_company_health, is defined in section 7) and 4 view-only tools, and 5 of the tools open an interactive view.

Transport and handshake:

- Streamable HTTP, stateless mode (no server-side session state), JSON responses.
- The server advertises the MCP Apps UI capability (`io.modelcontextprotocol/ui`).
- Views are resources with `ui://` URIs and MIME type `text/html;profile=mcp-app` . A tool links to its view through `_meta.ui.resourceUri` .
- Every tool returns both `structuredContent` (for the view) and a short text `content` (for Claude, and as the fallback in hosts without UI support).
- Each view declares a strict CSP in `_meta.ui.csp` : no external origins except our own API origin for PDF previews.
- View-only tools are marked visible to the app only, so Claude never sees them. They are called from the view, never by the model.
- Tool descriptions are 1–3 sentences, saying what the tool does and when to call it. They are part of the product's behavior, so they get reviewed like code.

Tool	Called by	Opens view	Purpose
`get_profile`	model	—	Profile, preferences, and the crawl query plan
`update_profile`	model	`profile-review`	Propose a profile change; the user approves it in the view
`submit_jobs`	model	—	Send crawled jobs; returns dedupe results + ghost base scores
`search_jobs`	model	—	Query the store for eligible, unscored, or recent jobs
`save_job_scores`	model	`shortlist`	Save fit + JD-quality scores in bulk; shows the ranked shortlist

Tool	Called by	Opens view	Purpose
`get_job`	model	`job-detail`	Full job, sightings timeline, signals, the user's score
`generate_cv`	model	`cv-studio`	Validate + render + compile a tailored CV and kit
`track_application`	model	—	Create or update one application
`list_applications`	model	`tracker`	The tracker board with follow-ups due
`ui_set_application_status`	view	—	Drag a card on the tracker
`ui_dismiss_job`	view	—	Hide a job from the shortlist
`ui_resolve_profile_change`	view	—	Approve or reject a proposed profile version
`ui_get_cv_pdf_url`	view	—	Signed, short-lived URL for the PDF preview

Schemas (`app/mcp/tools/schemas.py` ; Pydantic models generate the JSON Schema):

```
from datetime import date
from typing import Literal
from pydantic import BaseModel, Field, HttpUrl

class Preferences(BaseModel):
    target_titles: list[str] = Field(min_length=1, max_length=8)
    seniority_floor: Literal['junior','mid','senior'] = 'mid'
    allow_stretch: bool = True
    employment_types: list[Literal['full_time','contract']] = ['full_time']
    salary_floor_usd_month: int | None = None
    us_overlap_hours: float = Field(0, ge=0, le=8)
    country_code: str = Field(pattern=r'^[A-Z]{2}$')
    excluded_companies: list[str] = []

class CrawlPlan(BaseModel):
    queries: list[str]              # e.g. 'site:jobs.lever.co "backend
engineer" remote'
    seed_career_urls: list[HttpUrl] # known companies to re-visit
```

```
    fetch_budget: int = 30          # max page fetches per scan

class GetProfileOut(BaseModel):
    profile_version: int | None
    profile: dict | None            # MasterProfile, section 8
    preferences: Preferences | None
    crawl_plan: CrawlPlan

class SubmittedJob(BaseModel):
    title: str = Field(max_length=200)
    company_name: str = Field(max_length=200)
    company_domain: str | None = None
    location_text: str = Field(max_length=500)
    remote_type: Literal['remote','hybrid','onsite']
    employment_type: Literal['full_time','contract','part_time','internship']
| None = None
    salary_text: str | None = None      # raw; server parses it
    posted_at: date | None = None
    apply_url: HttpUrl
    source_url: HttpUrl
    source_kind: Literal['ats','careers_page','remote_board','search_result']
    jd_text: str = Field(min_length=200, max_length=40_000)

class SubmitJobsIn(BaseModel):
    jobs: list[SubmittedJob] = Field(min_length=1, max_length=50)

class SubmitJobResult(BaseModel):
    job_id: str
    is_new: bool
    sightings: int
    eligibility: Literal['worldwide','region_open','restricted','unclear']
    eligible_for_user: bool
    ghost_base_score: int
    needs_scoring: bool

class SearchJobsIn(BaseModel):
    only_unscored: bool = True
    since_days: int = Field(14, ge=1, le=90)
    title_query: str | None = None
    limit: int = Field(25, ge=1, le=50)

class JobScoreIn(BaseModel):
    job_id: str
```

```
    fit_band: Literal['match','stretch','reach','skip']
    fit_summary: str = Field(max_length=280)
    must_haves_met: list[dict]      # {requirement, evidence_bullet_id}
    gaps: list[dict]                # {requirement, transferable_from?,
severity: 'low'|'high'}
    jd_quality_flags:
list[Literal['no_responsibilities','no_team','talent_pool_wording','generic_t
emplate','named_hiring_contact']]

class SaveJobScoresIn(BaseModel):
    scores: list[JobScoreIn] = Field(min_length=1, max_length=50)

class GenerateCvIn(BaseModel):
    job_id: str
    template: Literal['classic','modern','compact'] = 'classic'
    content: dict                   # CvContent, section 8
    kit: dict                       # {cover_note, screening_answers[],
recruiter_dm, gap_report[]}

class TrackApplicationIn(BaseModel):
    job_id: str
    status:
Literal['saved','applying','applied','interviewing','offer','rejected','withd
rawn','ghosted']
    notes: str | None = None
    cv_id: str | None = None
```

Rules for tool behavior:

- `submit_jobs` is idempotent: submitting the same job twice adds a sighting, never a duplicate job.
- `save_job_scores` computes `ghost_score = clamp(base_score + jd_quality_points, 0, 100)` on the server. Claude never sends the final number.
- `generate_cv` returns immediately with `status: queued` and the view polls until the PDF is ready. Target compile time is under 8 seconds.
- Errors use MCP tool errors with a `code` (`quota_exceeded`, `validation_failed`, `not_found`) and a sentence Claude can relay to the user.
- Free plan limits are checked inside the tools: 3 CVs a month and 1 scan a day.

6. Discovery pipeline

Claude crawls and the server cleans up. Every job that `submit_jobs` receives goes through normalize → company match → dedupe → eligibility → sighting → signal recompute, all inside one transaction per job.

Crawl plan (built by `get_profile`). For each target title, the server emits queries against public ATS domains and remote boards:

```
ATS_SITES = ['boards.greenhouse.io', 'job-boards.greenhouse.io',
'jobs.lever.co',
             'jobs.ashbyhq.com', 'apply.workable.com']
BOARDS = ['remotive.com', 'remoteok.com', 'himalayas.app',
'weworkremotely.com']

def crawl_plan(prefs: Preferences, known: list[Company]) -> CrawlPlan:
    q = [f'site:{s} "{t}" remote' for t in prefs.target_titles for s in
ATS_SITES]
    q += [f'site:{b} {t}' for t in prefs.target_titles for b in BOARDS]
    q += [f'"{t}" remote "worldwide" OR "anywhere" job' for t in
prefs.target_titles]
    seeds = [c.careers_url for c in known if c.careers_url][:10]  # companies
seen before
    return CrawlPlan(queries=q[:24], seed_career_urls=seeds, fetch_budget=30)
```

Normalize (`discovery/normalize.py`):

- `title_norm`: lowercase; strip text in brackets and anything after `-` or `|` when it names a location or team; map synonyms (`sr` → `senior`, `swe` → `software engineer`, `eng` → `engineer`, `ml` → `machine learning`); collapse whitespace.
- `seniority`: regex on the title first (`intern|junior|jr|mid|senior|sr|staff|lead|principal`), then on the JD's years phrase (`(\d+)\+?\s*years`): 0–1 junior, 2–4 mid, 5–7 senior, 8+ staff.
- `jd_hash`: sha256 of the JD lowercased, with URLs, emails, dates, and whitespace runs removed, first 4,000 characters.
- Salary: parse `salary_text` with a regex for currency + range + period (`$4k-6k/month`, `USD 60,000 - 80,000 per year`, `PKR`). Convert to USD per month using a static rate table that's updated monthly in config. Leave fields null if the parse isn't confident.
- `on_company_ats`: true when `source_url` or `apply_url` is on one of `ATS_SITES` or on the company's own domain.

Company match: exact match on `domain` first. Otherwise trigram similarity on `name_norm` above 0.85. Otherwise create a new company. Set `ats_type` and `ats_slug` from the URL pattern (e.g. `jobs.lever.co/<slug>/...`).

Dedupe:

1. Exact: `(company_id, title_norm, jd_hash)` already exists → add a sighting and update `last_seen_at`.
2. Near: the same company, title trigram similarity above 0.9, and a JD of 3-shingle Jaccard similarity above 0.85 against any open job of that company → treat it as the same job and add a sighting.
3. Otherwise insert a new job. If the same `(company_id, title_norm)` has another job in the last 90 days, record it in `job_postings_history` (this is a repost).

Eligibility (`discovery/eligibility.py`): rules first, in this order. Stop at the first match.

Order	Pattern in location_text or JD	Result
1	`us only`, `must be (located|based) in the (us|united states)`, `us citizens`, `security clearance`, `w-2`	restricted (US)
2	`eu only`, `right to work in (the eu|uk)`, `uk only`, `canada only`	restricted
3	`worldwide`, `anywhere`, `global`, `work from anywhere`, `fully distributed`	worldwide
4	a region name (`emea`, `apac`, `mena`, `europe`, `asia`, `latam`) or a list of countries	region_open, with `allowed_countries` expanded from a region→country map
5	`remote` with a city or country next to it and none of the above	unclear

`eligible_for_user` is true for worldwide jobs, and for region_open jobs whose countries include the user's. For `unclear`, Claude is told in the tool result to read the JD and decide. The timezone requirement (`(\d+) hours? overlap`, `(est|pst|cet) hours`) fills `tz_min_overlap_hours` and is compared against `us_overlap_hours`.

Closing jobs: the nightly re-check (section 12) fetches each open job's `apply_url` with a plain GET. A 404 or 410, or a page that no longer contains the title, counts as a miss. Two misses in a row close the job.

7. Ghost-risk algorithm

The ghost score has two parts: a base score that the server computes from facts (`ghost/score.py`), and JD-quality points that Claude adds after reading the posting. Each part keeps its reasons, and every reason is shown to the user.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Reason:
    code: str
    points: int
    detail: str
    source_url: str | None = None

WEIGHTS = {  # app/ghost/weights.py, tuned later from outcomes
    'not_on_company_ats': 35,
    'repost_2plus_90d': 25,
    'repost_1_90d': 10,
    'age_over_45d': 15,
    'age_over_30d': 8,
    'company_shrinking': 15,
    'salary_listed': -10,
    'seen_by_many': -5,      # 3+ distinct users submitted it in the last 7
days
}
JD_FLAG_POINTS = {
    'no_responsibilities': 4, 'no_team': 3, 'talent_pool_wording': 6,
    'generic_template': 3, 'named_hiring_contact': -5,
}

def base_score(job, sig) -> tuple[int, list[Reason]]:
    r: list[Reason] = []
    if not sig.on_company_ats:
        r.append(Reason('not_on_company_ats', WEIGHTS['not_on_company_ats'],
                        'Not found on the company\'s own careers page or
ATS'))
    if sig.repost_count_90d >= 2:
```

```
        r.append(Reason('repost_2plus_90d', WEIGHTS['repost_2plus_90d'],
                        f'Reposted {sig.repost_count_90d} times in 90 days'))
    elif sig.repost_count_90d == 1:
        r.append(Reason('repost_1_90d', WEIGHTS['repost_1_90d'], 'Reposted
once in 90 days'))
    if sig.age_days is not None and sig.age_days > 45:
        r.append(Reason('age_over_45d', WEIGHTS['age_over_45d'], f'Posted
{sig.age_days} days ago'))
    elif sig.age_days is not None and sig.age_days > 30:
        r.append(Reason('age_over_30d', WEIGHTS['age_over_30d'], f'Posted
{sig.age_days} days ago'))
    if sig.company_health == 'shrinking':
        r.append(Reason('company_shrinking', WEIGHTS['company_shrinking'],
'Recent layoffs or hiring freeze reported'))
    if sig.salary_listed:
        r.append(Reason('salary_listed', WEIGHTS['salary_listed'], 'Salary
range is listed'))
    if sig.distinct_submitters_7d >= 3:
        r.append(Reason('seen_by_many', WEIGHTS['seen_by_many'], 'Found by
several users this week'))
    return max(0, min(100, sum(x.points for x in r))), r

def final_score(base: int, flags: list[str]) -> tuple[int, str]:
    s = max(0, min(100, base + sum(JD_FLAG_POINTS[f] for f in flags)))
    return s, 'low' if s < 30 else 'medium' if s < 60 else 'high'
```

When it runs: after every `submit_jobs` for the affected jobs, nightly for all open jobs (since age changes), and whenever a company's `health_flag` changes.

Repost count: the number of rows in `job_postings_history` for the same `(company_id, title_norm)` with `posted_at` (or first sighting) in the last 90 days, minus one.

Company health: when a job's company has no health check from the last 7 days, the `submit_jobs` result includes `needs_company_check: [company names]`. Claude then searches the news (layoffs, hiring freeze, funding) and calls `update_company_health` with a flag and a one-line note including the source URL. This is a 10th model-facing tool with input `{company_id, flag, note, source_url}`.

Tuning: a weekly job compares interview rates by ghost band from `applications`. If the Low band's rate isn't at least twice the High band's, flag the weights for review. Weights are never auto-changed.

8. CV engine

Claude never sends raw LaTeX. It sends structured `CvContent` that points at bullet IDs in the Master Profile. The server validates it against the profile, renders LaTeX from a template, and compiles a PDF. This makes fabrication checkable and LaTeX injection impossible.

Master Profile (`cv/schema.py` , stored encrypted):

```
class Bullet(BaseModel):
    id: str                         # 'b_<8 chars>', stable across versions
    text: str
    skills: list[str] = []          # skills this bullet is evidence for
    metrics: list[str] = []         # exact numbers/claims, e.g. '40%', '2M
rows'

class Role(BaseModel):
    id: str
    company: str
    title: str
    location: str | None
    start: str                      # 'YYYY-MM'
    end: str | None                 # None = present
    bullets: list[Bullet]

class Project(BaseModel):
    id: str; name: str; url: HttpUrl | None; bullets: list[Bullet]

class Education(BaseModel):
    id: str; school: str; degree: str; start: str; end: str | None; notes:
list[str] = []

class MasterProfile(BaseModel):
    name: str; headline: str; email: str; phone: str | None
    links: list[HttpUrl] = []
    summary_facts: list[str] = []   # raw facts a summary may draw on
    skills: list[str]               # canonical skill names
    roles: list[Role]; projects: list[Project] = []; education:
list[Education]
    certifications: list[str] = []
```

CvContent (sent by Claude to `generate_cv`):

```
class CvBullet(BaseModel):
    source_bullet_id: str           # must exist in the profile
    text: str = Field(max_length=260)

class CvSection(BaseModel):
    ref_id: str                     # role/project/education id
    bullets: list[CvBullet] = Field(max_length=6)

class CvContent(BaseModel):
    headline: str = Field(max_length=120)
    summary: str = Field(max_length=500)
    skills: list[str] = Field(max_length=24)
    sections_order:
list[Literal['experience','projects','education','skills','certifications']]
    experience: list[CvSection]
    projects: list[CvSection] = []
    education_ids: list[str]
```

Validation (`cv/validate.py`). Any failure rejects the request with `validation_failed` and a list of exact problems Claude can fix:

1. Every `ref_id` and `source_bullet_id` exists in the active profile version.
2. Every skill in `skills` is in `profile.skills` (case-insensitive, with an alias map such as `postgres` = `postgresql`).
3. **Numbers check:** every number or percentage in a rewritten bullet must appear in its source bullet's `text` or `metrics`.
4. **Entity check:** capitalized tech terms in a rewritten bullet (matched against a list of ~2,000 known technologies) must appear in the source bullet's text or skills.
5. The summary may only use numbers found in `summary_facts` or in some bullet's `metrics`.
6. Employers, titles, and dates are never in `CvContent`. The renderer takes them from the profile, so they can't be changed.
7. The PDF must fit 2 pages at most. The compile step counts pages, and if there are too many it returns `too_long` with the overflow in lines.

Render and compile:

- Jinja2 with LaTeX-safe delimiters (`\VAR{}`, `\BLOCK{}`). Every value goes through `latex_escape()` (escaping `& % $ # _ { } ~ ^ \`).

- Templates live in `cv/templates/{classic,modern,compact}.tex.j2`, use only standard packages, and are ATS-friendly: single column, real text (no text in images), and standard section names.
- Compile with `tectonic -X compile --untrusted --outdir <tmp>` in a subprocess with a 20-second timeout, no network access, a 512 MB memory limit, and a temp directory deleted afterward.
- Upload `cv.tex` and `cv.pdf` to object storage under `users/<user_id>/cvs/<cv_id>/`. The view gets a signed URL valid for 10 minutes.

Kit (also in `generate_cv`): `{cover_note ≤ 180 words, screening_answers: [{question, answer}], recruiter_dm ≤ 300 chars, gap_report: [{requirement, honest_framing}]}`. The cover note goes through the same numbers check as bullets.

9. Plugin: skills and commands

The plugin makes every user's Claude behave the same way. It holds the crawl procedure, both rubrics, and the CV rules. Ship it in `plugin/` and point users to install it alongside the connector.

`plugin/.claude-plugin/plugin.json`: name `remote-job-finder`, a description, and a version. It references the MCP server URL so installing the plugin also offers the connector.

`skills/daily-scan/SKILL.md`:

```
---
name: daily-scan
description: Find new remote jobs for the user, score them, and show the
shortlist. Use for "find jobs", "daily scan", "anything new?", or when run as
a scheduled task.
---
1. Call get_profile. If there is no active profile, ask for the user's CV and
stop.
2. Run the crawl_plan queries with web search. Open at most fetch_budget
pages in total.
   Prefer pages on ATS domains and company careers pages. Skip pages behind a
login.
   For LinkedIn/Indeed results, search for the same title + company on the
company's own site instead.
3. From each job page, extract the SubmittedJob fields exactly as written.
Never guess a salary or date.
   Skip pages with less than 200 characters of job description.
4. Call submit_jobs in batches of up to 50.
5. For each company in needs_company_check: search news from the last 6
months for layoffs,
   hiring freezes, or funding, then call update_company_health with a source
URL.
6. Call search_jobs(only_unscored=true). For each job, apply fit-rubric and
the JD flags in ghost-rubric.
   Skip jobs where eligible_for_user is false, unless eligibility is
'unclear' and the JD allows the user's country.
7. Call save_job_scores once with all scores. The shortlist view opens.
8. Reply in at most 3 lines: how many new jobs, how many Match/Stretch, and
the single best one with why.
```

`skills/fit-rubric/SKILL.md` (rules, abridged here only by the omitted frontmatter):

```
Score fit from the job's real requirements, never from the years number
alone.
- List must-haves (explicitly required) and nice-to-haves from the JD.
- For each must-have, cite the profile bullet id that proves it, or record it
as a gap.
- A gap is 'low' severity if a profile bullet shows a transferable skill
(same category:
  e.g. Rails→Django backend, Redshift→Snowflake); otherwise 'high'.
- match: no high gaps, and years asked ⩽ user's years + 1.
- stretch: at most 2 low gaps or 1 high gap, or years asked ⩽ user's years +
3.
- reach: anything harder that still shares the core domain.
- skip: different field, or below the user's seniority_floor.
- fit_summary: one sentence, specific, e.g. "Strong FastAPI + Postgres match;
no Kubernetes."
```

`skills/ghost-rubric/SKILL.md`:

```
You only judge the job description's quality. The server handles the ATS
check, reposts, age, salary, and company health.
Flag, when clearly true:
- no_responsibilities: no concrete duties listed.
- no_team: no team, product, or manager mentioned.
- talent_pool_wording: "future opportunities", "talent pool", "always
hiring", "pipeline".
- generic_template: the JD could fit any company; no product or stack
details.
- named_hiring_contact: a named recruiter or hiring manager is given (this
lowers risk).
Never flag on tone or writing quality alone.
```

`skills/cv-writer/SKILL.md`:

```
Build CvContent for generate_cv.
- Choose 3–6 bullets per recent role that best match the JD's must-haves;
drop the rest.
- Rewrite bullets in the JD's vocabulary. Keep every number exactly. Never
add a number, tool,
  employer, title, or date that is not in the source bullet. If the JD wants
something the profile lacks,
  put it in gap_report with an honest framing; never in the CV.
- Start bullets with a strong verb; one line each where possible; results
before tasks.
- Skills: only from profile.skills, JD-relevant ones first, max 24.
- If generate_cv returns validation_failed, fix exactly the listed problems
and retry once.
- If it returns too_long, drop the weakest bullets first.
- Kit: cover note ≤180 words, specific to the company; screening answers from
profile facts only;
  recruiter DM ≤300 characters.
```

Commands (`commands/*.md`, each a short prompt that triggers a skill):

Command	Does
`/jobs`	Runs daily-scan now
`/apply <job or number>`	Runs cv-writer for that job, opens CV studio, then offers to track it as applying
`/tracker`	Calls list_applications and opens the tracker view
`/profile`	Shows the profile; with a pasted CV, proposes an update

Scheduled scan: onboarding tells the user to create a daily scheduled task in Claude with the prompt `Run my daily job scan`. This is a user action; our server cannot create it for them.

10. UI

The design language is **"Briefing"**: calm, dense with evidence, and fast. Three signature details set it apart from every job board:

- the **sightings timeline**, which makes reposts and ghost jobs visible at a glance

- **provenance hover** in the CV studio, which ties every CV bullet to its proof and to the requirement it answers
- **evidence chips** on every claim

10.1 Design system (`packages/ui`)

Token group	Values
Color	Neutral gray scale (11 steps), one accent (indigo-600 light / indigo-400 dark), semantic `good` green, `warn` amber, `risk` red. All values are CSS variables. Views map host theme variables onto ours when the host provides them, and otherwise follow `prefers-color-scheme`.
Contrast	Body text 7:1 or better, secondary text 4.5:1 or better; semantic colors are never the only signal (always paired with an icon + label)
Type	Views: the system UI stack (no font download, matches the host). Web: Inter variable. Data: `font-variant-numeric: tabular-nums`. Scale 12 / 13 / 14 / 16 / 20 / 28 px, line height 1.45
Space	4 px grid; component padding 12 / 16; section gap 24
Shape	Radius 10 for cards, 6 for chips, 999 for pills; 1 px hairline borders, no drop shadows inside chat views
Motion	120 ms ease-out for hover/press, 180 ms for panels, spring (stiffness 400, damping 30) for drag. All motion becomes instant under `prefers-reduced-motion`
Icons	Lucide, 16 px, stroke 1.75

Core components: `JobCard`, `FitBadge` (Match / Stretch / Reach), `GhostMeter` (3-segment bar + band label + reasons popover), `EvidenceChip` (favicon + domain, opens the source), `SightingsTimeline`, `RequirementList` (met ✓ with the evidence bullet, gap

with severity), `PdfPreview`, `KanbanBoard`, `DiffRow`, `Skeleton`, `EmptyState`, `Toast`. Each is built on Radix primitives for focus management and ARIA, and has a Storybook story in both themes.

View runtime rules:

- Each view is one HTML file of 150 KB gzipped or less (React via `preact/compat` aliasing, if needed, to fit).
- Read data from the tool result's `structuredContent`. Call view-only tools through the MCP Apps SDK. Report content height to the host so it sizes the frame, and never show an inner scrollbar inline.
- Offer fullscreen where the host supports it: the shortlist, CV studio, and tracker.
- Updates are optimistic, with a rollback toast on failure.
- Every interactive element is reachable by Tab, and arrow keys move within lists and boards.

10.2 Shortlist view (opened by `save_job_scores`)

This view answers: which jobs should I look at today?

- **Header:** "14 new · 4 Match · 6 Stretch", the date, and a filter segmented control (All / Match / Stretch / Reach). High-ghost jobs sit behind a quiet "3 hidden as likely ghost jobs" link.
- **List:** `JobCard` rows sorted by band, then by ghost score ascending, then by recency. Each card shows the title, company with favicon, eligibility pill ("Worldwide", "EMEA incl. PK"), salary if listed, `FitBadge`, `GhostMeter`, and a one-line fit summary.
- **Interactions:** clicking a card or pressing Enter opens job detail (calls `get_job` via the host). `D` dismisses (`ui_dismiss_job`) with an undo toast. `A` starts an application by sending a follow-up message to Claude ("Apply to <job>").
- **States:** skeleton of 5 cards while loading; empty state "No new matches today — 37 jobs checked, 0 eligible" plus the top reason (usually location).

10.3 Job detail view (opened by `get_job`)

This view answers: is this job real, and do I fit?

- **Two columns on desktop, stacked on mobile.**
  - Left: the job, with must-have requirements highlighted inline in the JD (green when met, amber when a gap).

    - Right: three stacked panels.
        - **Fit:** the `RequirementList` with evidence bullet text on hover.
        - **Is it real?:** `GhostMeter` with every reason row (`+35 Not on company ATS`, `-10 Salary listed`) and its `EvidenceChip`.
        - **History:** `SightingsTimeline`.
- **Sightings timeline:** a 90-day horizontal axis. Each sighting is a dot colored by source kind, and each repost is a hollow diamond with a dashed connector to the earlier posting. Hovering a mark shows the date, source, and URL. This is the product's signature visual.
- **Actions:** primary "Build my CV for this" (sends a follow-up message to Claude), secondary "Save" and "Not for me".

10.4 CV studio (opened by `generate_cv`)

This view answers: is this CV right, and is it honest?

- **Split layout:** the left half shows the JD's must-haves as a checklist; the right half shows the live PDF preview (`ui_get_cv_pdf_url`).
- **Provenance hover:** hovering a CV bullet in the text outline (shown above the PDF) highlights the requirement it answers on the left and shows the original profile bullet underneath, with any changed words marked.
- **Gap report** panel, plus tabs for the cover note, screening answers, and recruiter DM. Each tab has a copy button that copies plain text.
- **Actions:** "Download PDF", "Download .tex", "Switch template" (classic / modern / compact; sends a message to Claude to regenerate), and "Mark as applied" (`track_application`).
- **States:** compiling shows a progress shimmer over a skeleton page. On failure, the exact validation problems appear, plus a "Ask Claude to fix" button that sends them as a message.

10.5 Tracker (opened by `list_applications`)

This view answers: where do I stand?

- A kanban with columns Saved → Applying → Applied → Interviewing → Offer, plus a collapsed "Closed" column (rejected, withdrawn, ghosted).
- Dragging a card calls `ui_set_application_status`. Keyboard users press Space to pick a card up and use arrow keys to move it.

- An amber "Follow up" badge appears on cards whose `follow_up_at` is today or earlier. The header shows "3 follow-ups due".
- Each card shows days since applying and the ghost band at the time of applying.

10.6 Profile review (opened by `update_profile`)

This view answers: is this what I want saved about me?

- A git-style diff of the proposed profile against the current one, grouped by role and project. Additions are green, removals red, edits shown word by word.
- Each change can be accepted or rejected, or everything can be accepted at once. The result calls `ui_resolve_profile_change`.
- Nothing becomes active until the user approves it here.

10.7 Web app (`apps/web`)

The web app handles what can't happen in chat. The server has no LLM, so creating the profile from a CV always happens in Claude, never on the website.

Screen	Purpose and content
Landing	One-line promise, a looping 20-second product video of the shortlist → job detail → CV studio, 3 proof points (ghost filter, eligibility, no fabrication), pricing, FAQ, and an "Add to Claude" button
Sign in	Passkey first; email one-time code as the fallback
Onboarding (4 steps)	1. Country + preferences form. 2. Add the connector (copy URL + 3-step guide with screenshots). 3. Install the plugin. 4. "Paste your CV into Claude and say: set up my profile", then set up the daily scheduled task. A progress checklist turns green as the server sees the first `get_profile` call and the first `submit_jobs` call.
Tracker	Full-screen version of the tracker view, using the same components
Insights	Applications by status, and interview rate by ghost band and fit band (charts, Pro only)

Screen	Purpose and content
Billing	Current plan, usage (CVs this month), upgrade or cancel through the payment provider's hosted checkout
Settings	Profile versions, active sessions and tokens with revoke, export all data (JSON + PDFs zip), delete account

Web performance budget: LCP under 1.8 s on a mid-range phone over 4G, CLS under 0.05, Lighthouse accessibility score of 100.

11. Auth, billing, security and privacy

We run our own OAuth 2.1 server so Claude can connect, our own passkey sign-in, and a merchant-of-record payment provider. Those plus an email sender for sign-in codes are the only outside dependencies.

OAuth for the connector (`app/auth`):

- Authorization code flow with PKCE (S256 only) and Dynamic Client Registration, so Claude can register itself as a client.
- Serve both `/.well-known/oauth-authorization-server` and `/.well-known/oauth-protected-resource` metadata. `/mcp` returns `401` with a `WWW-Authenticate` header pointing at the resource metadata.
- Access tokens: opaque, 1-hour lifetime, stored hashed. Refresh tokens: 30 days, rotated on every use, with the family revoked on reuse.
- Scopes: `jobs` (discovery + scores), `profile`, `cv`, `tracker`. The consent screen lists them in plain language.
- The consent page uses the same web sign-in (passkey or email code).

Web sign-in: passkeys (WebAuthn) as the primary method, and a 6-digit email code (10-minute expiry, 5 attempts) as the fallback. Sessions are HTTP-only, Secure, SameSite=Lax cookies.

Billing:

- Stripe doesn't onboard businesses registered in Pakistan. Use a merchant of record such as Paddle or Lemon Squeezy, which also handles global sales tax. Confirm that it pays out to Pakistan before integrating.

- Hosted checkout only; we never touch card data. A signed webhook (`/billing/webhook`, signature verified, idempotent on event ID) updates `users.plan`.
- Free plan: 1 scan a day, 3 CVs a month, 30-day job history. Pro: unlimited scans and CVs, full history, the Insights screen.

Security:

- `profiles.data` is encrypted with AES-256-GCM. The key comes from an environment variable and is rotated through key versions. It's decrypted only inside request scope and never logged.
- Rate limits per user (Redis token bucket): `submit_jobs` 20/min, `generate_cv` 10/hour, everything else 60/min.
- Input caps come from the Pydantic schemas, plus a 2 MB request body limit.
- The nightly re-check blocks SSRF: only http/https, DNS resolved first with private, loopback, and link-local IPs rejected, no redirects to other hosts, 10-second timeout, 2 MB maximum body, and an honest user agent with a contact URL.
- Tectonic runs sandboxed (section 8). All rendered text is escaped.
- Every view sets a strict CSP and makes no third-party requests.
- Every tool call writes `audit_log` (tool name, user, counts — never personal content).
- Dependency scanning and secret scanning run in CI.

Privacy:

- Settings offers "Export all data" (JSON + a zip of PDFs, ready within 1 minute) and "Delete account", which hard-deletes within 24 hours through a worker. Sightings the user submitted are kept but their link to the user is removed.
- Personal data is never used for anything other than serving that user. Shared job data contains no personal fields.
- The privacy policy is plain-language and linked on the consent screen.

12. Deployment, observability and testing

Everything runs from one `docker-compose.yml` locally and as containers in production, on any host that runs containers with managed Postgres and Redis.

Services: `api` (FastAPI + MCP, uvicorn, 2 workers), `worker` (Celery), `beat` (Celery beat), `web` (Next.js), `postgres`, `redis`. The API image includes the Tectonic binary with its package cache pre-warmed at build time, so compiles need no network.

Scheduled jobs (Celery beat, UTC):

Job	When	Does
`recheck_open_jobs`	01:00 daily	Plain GET on each open job's apply URL, at most 2 requests per second per domain; closes after 2 misses
`recompute_ghost_signals`	02:00 daily	Age and repost updates for all open jobs
`reset_cv_quota`	1st of month, 00:05	Resets `cv_quota_used`
`purge_deleted_users`	hourly	Hard-deletes accounts past their 24-hour window
`weekly_weight_report`	Monday 03:00	Interview rate by ghost band; writes a report row for review

Environments: `local`, `staging` (its own connector URL, used for the beta), `production`. Migrations run as a release step before new containers start.

Observability: structured JSON logs (`structlog`) with a request ID and tool name, and never personal content. Prometheus metrics: tool latency p50/p95, tool error rate by code, CV compile time, jobs submitted per day, dedupe rate, and the share of eligible jobs. Self-hosted Grafana dashboards. Alerts fire when the tool error rate exceeds 2% for 10 minutes or CV compile p95 exceeds 15 seconds.

Testing:

Layer	Tooling	Must cover
Unit	pytest	normalize, dedupe, eligibility (a 150-case golden table of real location strings), salary parsing, ghost score, all 7 CV validation rules
Integration	pytest + testcontainers Postgres	every tool end to end, idempotent `submit_jobs`, user scoping (user A can never read user B's rows)

Layer	Tooling	Must cover
MCP contract	MCP Inspector CLI in CI	every tool lists with a valid schema; `ui://` resources resolve with the right MIME type
CV	pytest	each template compiles; a 2-page limit fixture; injection strings (`\input{/etc/passwd}`) come out escaped
Views	Vitest + Testing Library; Playwright with a mock host	each view renders from fixture `structuredContent`; keyboard paths; axe accessibility scan with zero violations; visual snapshots in light/dark at 360 / 768 / 1280 px
Web	Playwright + Lighthouse CI	onboarding flow; performance budgets from section 10.7
Eval	Manual, weekly during beta	20 real jobs scored by Claude with the plugin versus human judgment; at least 80% band agreement

CI (GitHub Actions): lint → type check → unit → integration → contract → views → build images. Merging to `main` deploys to staging, and a tagged release deploys to production.

13. Build milestones

Seven milestones, each shippable and testable on its own. Claude Code works through them in order and ticks off every box before moving on. M3 is the first point where the product does something real for its first user.

M0 · Scaffold

- [ ] Repo matches the tree in section 3; `CLAUDE.md` written
- [ ] `docker compose up` starts all services; `/healthz` returns 200
- [ ] CI runs lint, type check, and tests on every push

M1 · Data and discovery core

- [ ] Migration 001 equals the DDL in section 4

- [ ] Normalize, company match, dedupe, and eligibility pass their golden tests (150 location strings with at least 95% correct)
- [ ] The ghost base score matches section 7 on 20 hand-built fixtures
- [ ] Submitting the same job twice yields 1 job and 2 sightings

M2 · MCP server and OAuth

- [ ] OAuth 2.1 + PKCE + DCR work; metadata endpoints are served
- [ ] All 10 model-facing tools respond with valid schemas in the MCP Inspector
- [ ] On staging, the connector adds to claude.ai as a custom connector and `get_profile` succeeds after sign-in
- [ ] Cross-user access tests pass

M3 · Plugin and first real scan

- [ ] Plugin installs; `/jobs` runs daily-scan end to end
- [ ] With a real profile (Abdullah's), one scan stays within its fetch budget, submits 20 or more jobs, and saves scores
- [ ] Of the jobs shown as Low ghost risk, a manual check finds at least 90% live on the company's own site
- [ ] A daily scheduled task runs the scan unattended for 3 days in a row

M4 · CV engine

- [ ] All 3 templates compile in under 8 s at p95 and fit 2 pages
- [ ] All 7 validation rules are enforced; a fabricated number or skill is rejected with a precise message
- [ ] `/apply` produces a PDF, a .tex file, and a full kit for 5 real jobs, each reviewed by a human as truthful

M5 · Design system and in-chat views

- [ ] `packages/ui` components have stories in both themes
- [ ] All 5 views render in Claude from real tool results, including the sightings timeline and provenance hover
- [ ] Accessibility scan shows zero violations; full keyboard paths work; each view is 150 KB gzipped or less

- [ ] Text fallbacks read well in hosts without UI support

M6 · Web app, billing, beta

- [ ] Landing, sign-in, onboarding, tracker, insights, billing, and settings shipped to the section 10.7 budgets
- [ ] Merchant-of-record checkout and webhook switch plans correctly; free-plan limits are enforced in the tools
- [ ] Export and delete work end to end
- [ ] 10–20 beta users onboarded; the weekly eval shows at least 80% band agreement
- [ ] Connector directory submission sent

Sources

- MCP Apps overview: `ui://` resources, `_meta.ui.resourceUri`, sandboxed views
- MCP Apps SDK and Claude Code plugin
- Interactive connectors in Claude: MCP Apps available on Claude web, desktop, and mobile
- Building MCP Apps (Laravel write-up): UI capability, MIME type, view-only tools